`std::find` is a useful algorithm to consider next, because it's one of several algorithms that can complete without every element having been processed. For example, if the first element in the range matches the search criterion, there's no need to examine any other elements. As you'll see shortly, this is an important property for performance, and it has direct consequences for the design of the parallel implementation. It's a particular example of how data access patterns can affect the design of your code (section 8.3.2). Other algorithms in this category include `std::equal` and `std::any_of`.

If you were searching for an old photograph through the boxes of keepsakes in your attic with your wife or partner, you wouldn't let them continue searching if you found the photograph. Instead, you'd let them know you'd found the photograph (perhaps by shouting, "Found it!"), so that they could stop searching and move on to something else. The nature of many algorithms requires that they process every element, so they have no equivalent to shouting, "Found it!" For algorithms such as `std::find` the ability to complete "early" is an important property and not something to squander. You therefore need to design your code to make use of it—to interrupt the other tasks in some way when the answer is known, so that the code doesn't have to wait for the other worker threads to process the remaining elements.

If you don't interrupt the other threads, the serial version may well outperform your parallel implementation, because the serial algorithm can just stop searching and return once a match is found. If, for example, the system can support four concurrent threads, each thread will have to examine one quarter of the elements in the range, and our naïve parallel implementation would thus take approximately one quarter of the time a single thread would take to check every element. If the matching element lies in the first quarter of the range, the sequential algorithm will return first, because it doesn't need to check the remainder of the elements.

One way in which you can interrupt the other threads is by making use of an atomic variable as a flag and checking the flag after processing every element. If the flag is set, one of the other threads has found a match, so you can cease processing and return. By interrupting the threads in this way, you